Compte rendu du Mini-Projet

Algorithmique Avancée

Ales Amazigh Ferhani
Numéro étudiant : 22203258

1 Introduction

L'objectif de ce mini-projet est de concevoir une application permettant de visualiser et de comparer le comportement d'algorithmes de recherche de plus court chemin sur un environnement réel, à savoir une image numérique. Contrairement aux cartes abstraites utilisées dans les travaux pratiques, ce projet s'appuie sur une image chargée par l'utilisateur, dont les pixels sont interprétés comme un terrain traversable.

L'application est développée en Java avec une interface graphique JavaFX. Elle permet de sélectionner interactivement un point de départ et un point d'arrivée directement sur l'image, puis d'observer en temps réel la progression de l'algorithme de Dijkstra, ainsi que le chemin final calculé.

2 Présentation du problème

Le problème étudié consiste à déterminer un chemin optimal entre deux pixels d'une image considérée comme une grille bidimensionnelle. Chaque pixel représente une position possible dans l'environnement, et le passage d'un pixel à un autre est associé à un coût dépendant de la différence d'intensité entre les pixels voisins.

Les pixels de l'image sont ainsi assimilés aux sommets d'un graphe, tandis que la relation de voisinage définit les arêtes. Le poids de chaque arête correspond à la variation d'intensité entre les deux pixels qu'elle relie, ce qui permet de pénaliser les transitions fortement contrastées.

Ce problème peut alors être formulé comme un problème de plus court chemin dans un graphe pondéré, ouvrant la voie à l'utilisation de l'algorithme de Dijkstra pour la recherche d'un chemin de coût minimal.

3 Modélisation et construction du graphe

Dans notre implémentation, l'image chargée est modélisée sous la forme d'un graphe pondéré non orienté, construit à partir de la grille de pixels.

- **Sommets** : chaque pixel de l'image correspond à un sommet du graphe.
- **Arêtes** : deux sommets sont reliés si les pixels correspondants sont voisins selon une connexité en 4 ou en 8 directions.

- **Poids** : le coût du déplacement entre deux pixels voisins A et B est défini par :

$$\text{Poids}(A, B) = c(A, B) + |\text{Intensité}(A) - \text{Intensité}(B)|$$

$$c(A, B) = \begin{cases} 1 & \text{si } A \text{ et } B \text{ sont voisins horizontaux ou verticaux,} \\ \sqrt{2} & \text{si } A \text{ et } B \text{ sont voisins diagonaux.} \end{cases}$$

Signification des termes de la formule

- $c(A, B)$ représente le coût unitaire associé à une transition locale entre deux pixels voisins, indépendamment de leur intensité.
- Intensité(A) et Intensité(B) correspondent aux niveaux de gris des pixels associés aux sommets A et B, avec des valeurs comprises entre 0 (noir) et 255 (blanc).

Cette pondération permet de modéliser un compromis entre la longueur du chemin, mesurée en nombre de transitions entre pixels voisins, et l'homogénéité visuelle du parcours, favorisant les zones de faible contraste. Le problème se ramène ainsi à un calcul de plus court chemin dans un graphe pondéré à coûts strictement positifs, pour lequel l'algorithme de Dijkstra est particulièrement adapté.

4 Implémentation de l'algorithme de Dijkstra

L'algorithme de Dijkstra est utilisé dans ce projet comme unique méthode de recherche de plus court chemin. Il permet de déterminer un chemin de coût minimal entre un pixel de départ et un pixel d'arrivée dans le graphe pondéré modélisant l'image.

L'algorithme procède par une exploration progressive et uniforme du graphe. À chaque itération, il sélectionne parmi les sommets non encore traités celui dont la distance courante depuis la source est minimale. Cette stratégie ne repose sur aucune information directionnelle globale et conduit à une exploration concentrique autour du point de départ.

Cette exploration exhaustive garantit l'obtention d'un chemin optimal, au prix d'une exploration potentiellement étendue de l'image, en particulier lorsque la cible est éloignée ou lorsque le graphe contient de nombreuses zones de faible contraste.

Utilisation de la `PriorityQueue`

L'implémentation repose sur une `PriorityQueue` permettant d'extraire efficacement le sommet ayant la distance minimale depuis la source. Cette structure de données joue un rôle central dans l'algorithme de Dijkstra en garantissant que les sommets sont traités dans l'ordre croissant de leur distance, assurant ainsi la correction et l'efficacité de l'algorithme.

Les distances minimales depuis le sommet source sont stockées dans un tableau, mis à jour par des opérations de relaxation lors de l'examen des sommets voisins. Un tableau de prédécesseurs est également maintenu afin de permettre la reconstruction du chemin optimal une fois l'algorithme terminé.

Figure 1: Début de l'exploration

Les premiers sommets visités apparaissent autour du pixel de départ, illustrant l'initialisation de l'exploration à partir de la source.

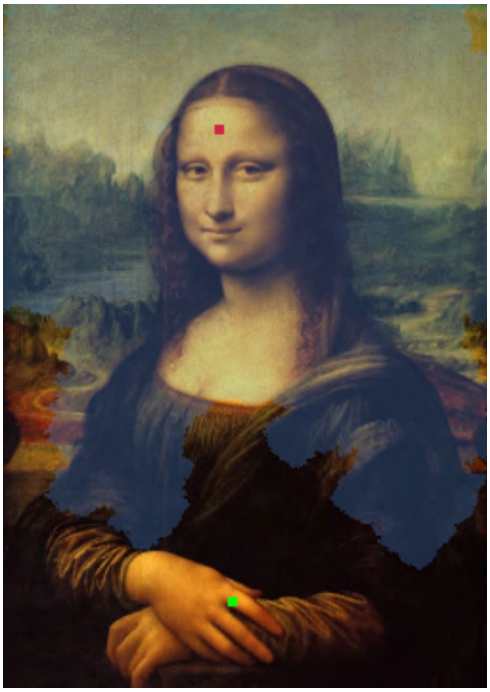

Figure 2: Exploration intermédiaire

La zone de sommets visités s'étend progressivement autour du point de départ, traduisant le caractère uniforme et non directionnel de l'algorithme.

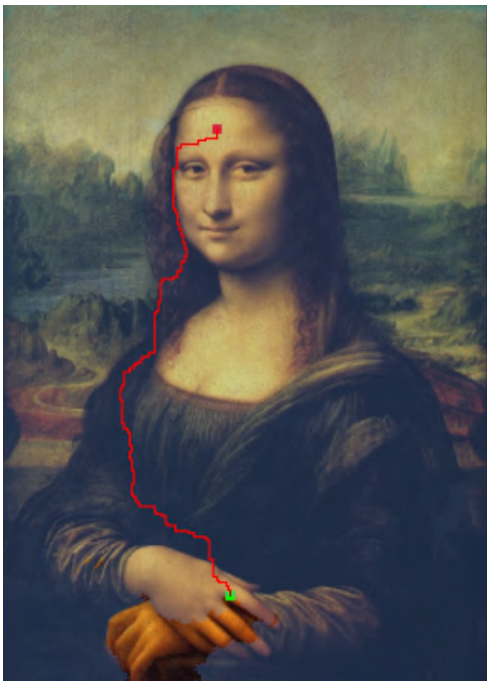

Figure 3: Fin d'exploration

L'extension de la zone explorée illustre le nombre de sommets que l'algorithme doit examiner afin de garantir l'optimalité du chemin, en fonction de la position du pixel d'arrivée et des variations de coût présentes dans l'image.

5 Choix algorithmique et justification

Dans ce projet, l'algorithme de Dijkstra a été retenu comme unique méthode de recherche de plus court chemin. Ce choix est directement lié au modèle de coût adopté pour la construction du graphe.

Une première version de l'application intégrait également l'algorithme A*, implémenté de manière fonctionnelle afin de comparer son comportement à celui de Dijkstra. Toutefois, après une analyse plus approfondie du modèle de coût et à la suite de discussions avec d'autres étudiants, il est apparu que les informations disponibles pour guider la recherche étaient exclusivement locales.

Le coût associé à chaque transition entre deux pixels voisins repose en effet sur un coût unitaire constant et sur la différence d'intensité entre les pixels, sans faire intervenir de distance globale ou de métrique permettant d'estimer de manière fiable le coût restant jusqu'au pixel d'arrivée. Dans ce contexte, la définition d'une heuristique admissible et pertinente s'avère délicate, l'algorithme A* tendant alors à se comporter de manière similaire à Dijkstra, sans gain significatif en termes de performances.

Le choix de conserver uniquement l'algorithme de Dijkstra dans la version finale permet ainsi de garantir l'optimalité du chemin calculé tout en assurant une parfaite cohérence entre le modèle de coût, la modélisation du graphe et l'algorithme employé. Cette approche privilégie la clarté conceptuelle et la robustesse du modèle plutôt que l'introduction d'une heuristique artificielle peu représentative du problème étudié.

6 Visualisation et analyse des performances

L'exécution de l'algorithme de Dijkstra est accompagnée d'une visualisation animée permettant d'observer concrètement son comportement sur l'image. Les sommets visités au cours de l'exécution sont affichés en bleu, tandis que le chemin final de coût minimal est représenté en rouge une fois le calcul terminé.

Cette visualisation met en évidence l'étendue de la zone explorée en fonction de la position du pixel d'arrivée et de la distribution des coûts dans l'image. Dans les zones de faible contraste, l'exploration tend à s'étendre davantage, tandis que des variations d'intensité plus marquées influencent la forme du chemin calculé.

Un tableau de statistiques complète cette visualisation et fournit des indicateurs quantitatifs tels que le nombre de sommets explorés, la longueur du chemin obtenu et son coût total. Ces données permettent d'analyser l'impact du modèle de coût sur le résultat final et d'illustrer le compromis entre longueur du parcours et homogénéité visuelle du chemin.

Le chemin final est reconstruit à partir du tableau des prédécesseurs et visualisé directement sur l'image.

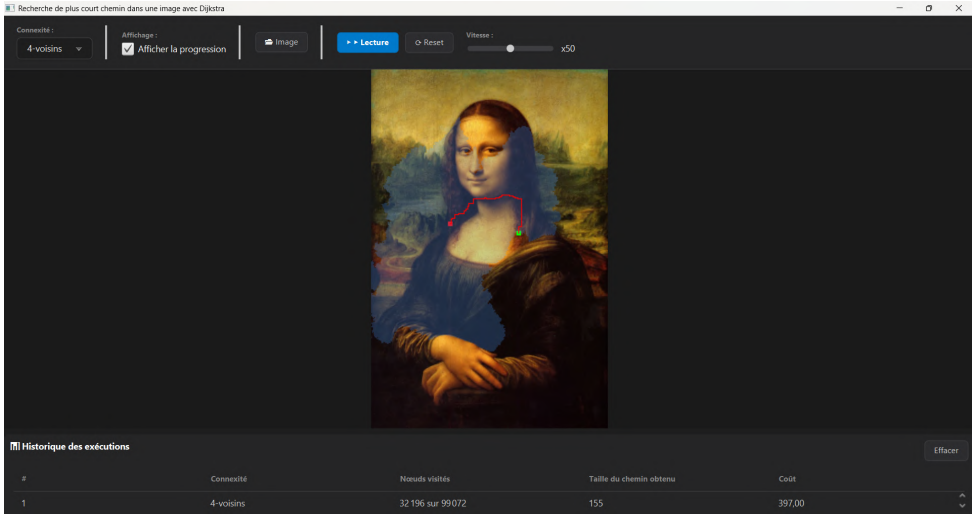

Figure 4: Chemin de coût minimal obtenu à l'issue de l'algorithme de Dijkstra

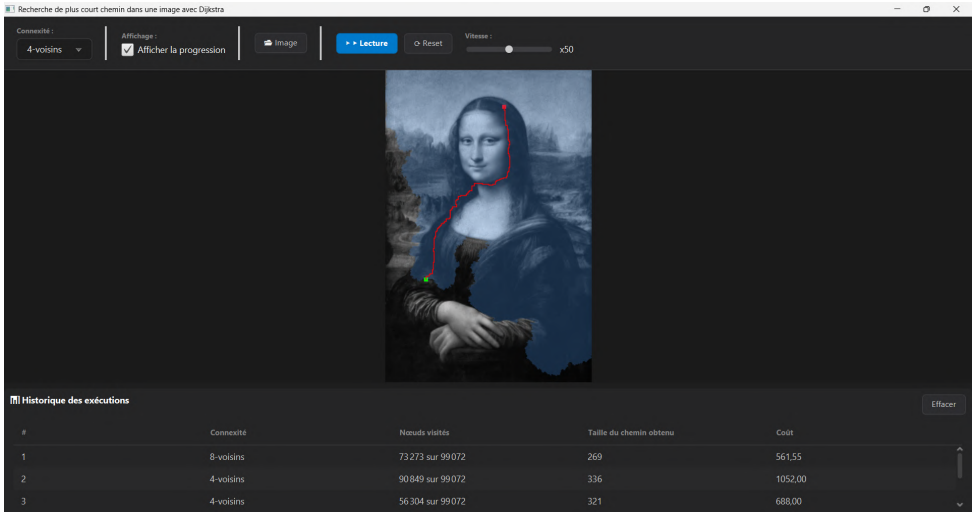

Figure 5: Test de l'algorithme de Dijkstra sur une image en niveaux de gris

Comme illustré à la Figure 5, la faible variation d'intensité entre pixels conduit à un chemin principalement influencé par la longueur du parcours.

7 Complexité algorithmique de l'algorithme de Dijkstra

La complexité de l'algorithme de Dijkstra dépend à la fois de la structure du graphe et des structures de données utilisées pour son implémentation. Dans ce projet, le graphe modélisant l'image est représenté sous la forme d'une liste d'adjacence et l'algorithme repose sur l'utilisation d'une PriorityQueue pour la sélection du sommet de distance minimale.

Soit V le nombre de sommets du graphe et E le nombre d'arêtes. Dans le cas général, l'algorithme de Dijkstra implémenté avec une file de priorité présente une complexité en temps de l'ordre de $\mathcal{O}((V+E)\log V)$. Chaque sommet est inséré au plus plusieurs fois dans la file de priorité, et chaque extraction ou insertion s'effectue en temps logarithmique.

Dans le contexte spécifique de ce projet, chaque pixel de l'image correspond à un sommet du graphe, ce qui implique $V = W \times H$, où W et H désignent respectivement la largeur et la hauteur de l'image. Le nombre d'arêtes est proportionnel au nombre de sommets, chaque pixel étant relié à un nombre constant de voisins (4 ou 8), ce qui conduit à $E = \mathcal{O}(V)$.

La complexité temporelle de l'algorithme est donc de l'ordre de $\mathcal{O}(V \log V)$, soit $\mathcal{O}((W \times H)\log(W \times H))$ dans le cas d'une image de taille $W \times H$. En pratique, cette complexité se traduit par un temps de calcul croissant avec la taille de l'image, ce qui explique l'étendue de l'exploration observée sur les images de grande dimension.

En termes de complexité mémoire, l'algorithme nécessite le stockage des distances, des prédécesseurs et de l'ensemble des sommets visités, ce qui représente une complexité en espace de l'ordre de $\mathcal{O}(V)$.

8 Difficultés rencontrées

Les principales difficultés rencontrées au cours du projet concernent la modélisation de l'image sous forme de graphe pondéré et la définition d'une fonction de coût adaptée au problème étudié. Il a notamment fallu trouver un compromis pertinent entre la longueur du chemin et l'homogénéité visuelle du parcours, en combinant un coût unitaire de transition et la différence d'intensité entre pixels voisins.

La construction du graphe a également nécessité une attention particulière quant à la gestion du voisinage des pixels, en distinguant correctement les déplacements horizontaux, verticaux et diagonaux, tout en garantissant la cohérence des poids associés à chaque arête.

Par ailleurs, un effort important a été consacré à la structuration de l'application afin de séparer clairement la logique algorithmique de la représentation graphique. Cette séparation a permis de faciliter le débogage, d'améliorer la lisibilité du code et de rendre l'application plus évolutive, notamment en ce qui concerne la visualisation animée de l'exploration et du chemin final.

9 Temps passé

Le temps total consacré à ce TP est estimé à environ 16 heures. Il se répartit de la manière suivante :

- Analyse du sujet et compréhension des consignes : environ 1h30.
- Conception et implémentation de l'algorithme de Dijkstra : environ 1h30.
- Implémentation initiale d'une version alternative de recherche de chemin et analyse de sa pertinence dans le cadre du projet : environ 3h.
- Phases de test et d'observation du comportement de l'algorithme : environ 2h.
- Correction de bugs et ajustements de l'implémentation : environ 2h.
- Rédaction du compte rendu : environ 6h.

Une part significative de ce temps a été consacrée à la réflexion sur la modélisation du problème et sur la définition du modèle de coût, afin de garantir la cohérence entre la structure du graphe, l'algorithme retenu et les résultats observés lors de l'exécution de l'application.

10 Applications réelles de la recherche de plus court chemin sur une image

La recherche de plus court chemin sur une image ne se limite pas à un exercice algorithmique abstrait. Elle trouve de nombreuses applications concrètes dans des domaines où l'image constitue une représentation de l'environnement ou des données à analyser.

En imagerie médicale, ce type d'approche est utilisé pour l'extraction de structures anatomiques à partir d'images issues de scanners ou d'IRM. La détermination automatique de contours, tels que des vaisseaux sanguins, des nerfs ou des frontières entre tissus, peut être formulée comme un problème de plus court chemin dans un graphe d'image. Le coût associé aux transitions entre pixels permet alors de privilégier les zones homogènes et d'éviter les ruptures brutales d'intensité, de manière similaire au modèle de coût utilisé dans ce projet.

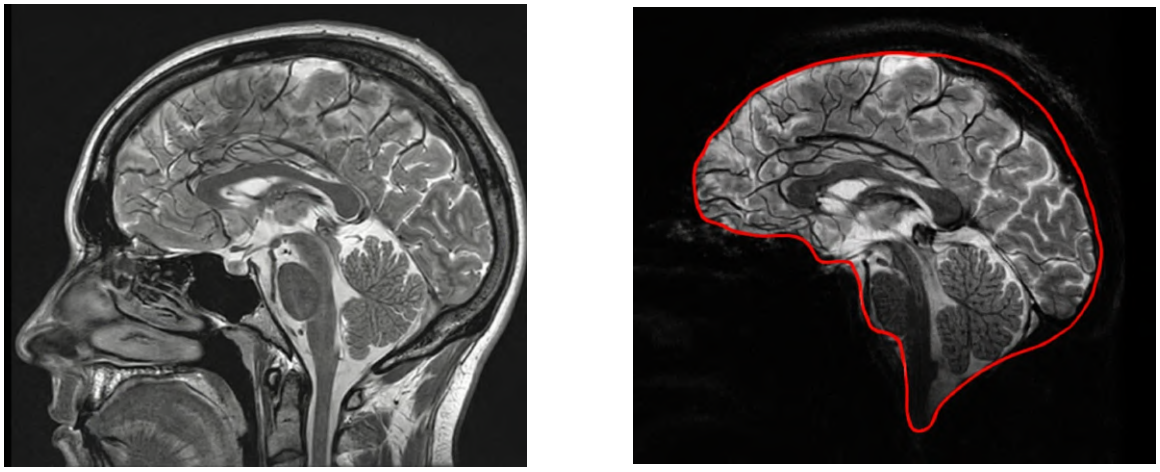

Figure 6: Délimitation du cerveau sur une image médicale

Dans le domaine de la vision par ordinateur, la recherche de chemin optimal est également employée pour la segmentation d'images, le suivi de trajectoires ou l'analyse de formes. Un chemin de coût minimal peut par exemple représenter une frontière optimale entre deux régions d'une image ou une trajectoire visuelle cohérente au sein d'une séquence d'images.

En robotique et en navigation autonome, les images issues de capteurs peuvent être interprétées comme des cartes de coût, où chaque pixel représente une zone plus ou moins favorable au déplacement. La recherche de plus court chemin permet alors de planifier des trajectoires sûres en tenant compte des obstacles et des zones à éviter, en s'appuyant sur une modélisation locale des coûts.

Enfin, des applications existent également en géographie et en traitement d'images satellites, notamment pour la détection de routes, de rivières ou de frontières naturelles. Dans ces contextes, l'image sert de support à une représentation discrète du terrain, et la recherche de chemin optimal permet d'extraire automatiquement des structures linéaires pertinentes.

Figure 7: Détection de chemins naturels sur une image géographique (ici la Seine)

Ces exemples montrent que la recherche de plus court chemin sur une image constitue un outil général et puissant, applicable à des problématiques variées dès lors que l'image peut être modélisée comme un graphe pondéré.

11 Conclusion

Ce mini-projet a permis de mettre en application de manière concrète les notions d'algorithmique avancée liées aux graphes et à la recherche de plus court chemin. La modélisation d'une image numérique sous la forme d'un graphe pondéré et l'utilisation de l'algorithme de Dijkstra ont permis de comprendre en profondeur le fonctionnement d'un algorithme de recherche de chemin optimal dans un contexte réaliste.

La visualisation de l'exploration et du chemin final sur une image réelle s'est avérée particulièrement instructive. Elle constitue un support pédagogique pertinent pour observer le comportement de l'algorithme, analyser l'impact du modèle de coût et comprendre le compromis entre la longueur du chemin et l'homogénéité visuelle du parcours. Ce projet a ainsi mis en évidence l'importance des choix de modélisation, notamment dans la définition des coûts associés aux transitions entre pixels.

Au-delà de l'aspect technique, ce projet m'a permis de mieux percevoir l'utilité concrète de la recherche de plus court chemin dans des applications réelles. Les exemples abordés, tels que l'imagerie médicale, la vision par ordinateur ou l'analyse d'images géographiques, montrent que ces algorithmes jouent un rôle central dans des domaines variés où l'image peut être interprétée comme une représentation discrète de l'environnement.

Le thème du projet, consacré à la recherche de plus court chemin, m'a particulièrement intéressé car il se rapproche directement de ce que je souhaite faire plus tard dans le domaine de l'intelligence artificielle. La planification de trajectoire, l'optimisation et la prise de décision sont en effet des problématiques fondamentales en IA, que ce soit pour les véhicules autonomes, la robotique mobile ou d'autres systèmes intelligents. Ce projet a renforcé mon intérêt pour ces thématiques en mettant en lumière leur lien étroit avec les algorithmes de graphes étudiés en cours.

En conclusion, ce mini-projet a constitué une expérience formatrice, à la fois sur le plan algorithmique et sur le plan applicatif. Il a permis de consolider les connaissances théoriques vues en cours, tout en illustrant leur pertinence dans des problématiques concrètes et motivantes.